

E.E.S.T. N° 2 · SISTEMA ENRO

Manual del programador Sistema ENRO

Arquitectura, API, datos y mantenimiento

`Laravel 10 · PHP 8.2 · MySQL 8`
`React 19 · Vite 6 · Node 22 · Docker`

Documento técnico interno

Sistema v2.0 · Edición noviembre 2026

Para quien mantiene, extiende o despliega el sistema.

Cómo leer este manual

Este documento explica cómo está construido el Sistema ENRO, dónde vive cada cosa y qué reglas no conviene romper al modificarlo. Está pensado para una persona que ya programa y que necesita incorporarse rápido al proyecto, sin tener que reconstruir la arquitectura leyendo archivo por archivo.

Alcance y código activo

El sistema activo es backend/ (Laravel) y frontend/ (React); todo cambio se hace ahí. Las viejas copias sistema-escuela-back/ y sistema-escuela-front/ se eliminaron el 2026-11-04: estaban vacías y no las referenciaba nada. Quedan dos carpetas _backup_* con el estado previo a los arreglos de escala; son red de seguridad, no código para editar. Antes de borrar o mover algo, confirmá con el equipo.

Contenido

Capítulo	Qué resuelve
01 · Arquitectura	Stack, capas, flujo de una petición y decisiones de diseño.
02 · Puesta en marcha	Docker, instalación local, variables y datos iniciales.
03 · Repositorio	Estructura de carpetas, convenciones y respaldos.
04 · Backend	Rutas, middleware, controladores, modelos y soporte.
05 · Base de datos	Tablas, relaciones, índices, migraciones y seeders.
06 · API y sesiones	Token propio, permisos, límites, errores e impresión.
07 · Frontend	Rutas, guards, cliente API, caché y refresco automático.
08 · Módulos	Alumnos, asistencia, notas, cursos, documentos y backups.
09 · Reglas de negocio	Ámbitos, asistencia, calificaciones y promoción.
10 · Pruebas	PHPUnit, ESLint, build y criterios de verificación.
11 · Despliegue	Docker en Linux, OPcache, backups, logs y checklist de producción.
12 · Depuración	Síntomas frecuentes, causa probable y qué revisar.
13 · Contribución	Convenciones, checklist y errores que no hay que repetir.
Anexos	Comandos, archivos para leer primero y checklist del primer día.

Convenciones

Rutas: se escriben desde la raíz del repositorio, por ejemplo backend/app/Dominios.

Comandos: se muestran para Docker (producción) y para Laragon en Windows (desarrollo local). En Laragon los binarios no están en el PATH, así que se usa la ruta completa del ejecutable.

Código: los bloques oscuros son extractos reales o simplificados del repositorio.

Roles: admin, directivo, jefe_preceptores, preceptor, prosecretaria y profesor.

Regla de oro: primero entendé el flujo, después tocá el código; nunca pruebes en producción.

Conocimientos previos

HTTP, JSON y el modelo request/response. No hace falta ser experto en redes.

Laravel: rutas, middleware, Eloquent, validación y migraciones.

React: componentes, hooks, estado y rutas del lado del cliente.

SQL: SELECT, JOIN, índices y transacciones. MySQL 8 es el motor real.

Si falta alguno de estos temas, no es bloqueante: el manual señala dónde mirar y el código está comentado en español. Lo que no se puede saltear es la lectura de las reglas de negocio del capítulo 9.

01 Arquitectura general

Stack, capas y recorrido de una petición.

El Sistema ENRO es una aplicación web monolítica con API JSON y SPA. No usa microservicios ni una cola de trabajos: el navegador habla con Laravel y Laravel habla con MySQL. Esa simplicidad es deliberada y explica muchas decisiones del código.

Stack real

Capa	Tecnología	Uso en el proyecto
Backend	Laravel 10.48 / PHP 8.2	API, reglas, autenticación, PDF y Excel. En Docker y producción es 8.2; en desarrollo local (Laragon) corre 8.3.
Frontend	React 19.1 / Vite 6.3	SPA con módulos por dominio y code-splitting.
Ruteo	React Router 7.9	Rutas públicas y protegidas con guards.
Base de datos	MySQL 8.0 / utf8mb4	Datos escolares, auditoría y sesiones.
Servidor	Nginx + PHP-FPM	Front controller, compresión y límite de subida.
PDF	barryvdh/laravel-dompdf	Planillas, listados y constancias imprimibles.
Excel	PhpSpreadsheet 5.10	Planilla oficial de calificaciones y padrón de alumnos (.xlsx).

Capas y responsabilidades

Presentación: frontend/src/modulos. Cada módulo tiene pantallas, componentes y estilos. No guarda reglas de negocio: las pide al backend.

Red: frontend/src/compartido/lib/api.js. Es la única puerta de salida: token, errores, caché e invalidación.

API: backend/routes. Define rutas públicas, protegidas y grupos por rol.

Aplicación: backend/app/Dominios. Un proceso por carpeta: Alumnos, Asistencia, Calificaciones, Cursos, Documentos, Profesores, Sistema y Usuarios.

Soporte: backend/app/Support. Reglas y ayudas compartidas: permisos, jerarquía, planilla, sello, paginación.

Persistencia: Eloquent y Query Builder sobre MySQL. Las migraciones son la fuente de verdad del esquema.

Recorrido de una petición

1. El componente React llama a apiGet, apiPost, apiPut o apiDelete.
2. api.js arma la URL, agrega Accept, X-Requested-With y Authorization: Bearer.
3. Nginx recibe la petición y la pasa a PHP-FPM; Laravel resuelve la ruta de api.php.
4. AuthToken valida el token contra api_sesiones (o users.api_token como respaldo) y exige status active.
5. EnsureRole aplica la jerarquía; el controlador vuelve a validar el ámbito del curso o alumno.
6. El controlador valida el body, ejecuta la operación y registra la actividad con ActivityLogger.
7. Laravel devuelve JSON. Si es una escritura, api.js invalida las copias y marca el cambio propio.

Mapa en 60 segundos

El navegador carga una SPA de React. Cada pantalla pide datos a /api con un único cliente HTTP que agrega el token. Laravel valida token, rol y ámbito, ejecuta el controlador y consulta MySQL. Las vistas imprimibles y los Excel salen por rutas web separadas, también autenticadas.

```
Navegador (React + Vite)
   -> fetch /api/...  (Authorization: Bearer <token>)
      -> Nginx :8000 -> PHP-FPM -> Laravel
         -> auth.token -> EnsureRole -> Controlador
            -> Dominio / Eloquent -> MySQL 8
         <- JSON (o PDF/XLS por /calificaciones/planilla, /print/...)
```

Ese recorrido se repite en casi todas las funciones: si entendés dónde se corta, ubicás un 401, 403, 422 o 500 sin adivinar.

Dos capas de permisos, no una

El middleware role deja pasar a un grupo de roles. Pero el controlador igual verifica si el usuario está asignado al curso o al alumno. Nunca reemplaces esa segunda validación por una condición de frontend: el navegador no es confiable.

Decisiones que conviene entender

Token propio, no Sanctum: la sesión vive en api_sesiones, con un registro por equipo. Se eligió para poder listar y cerrar equipos sin depender de cookies.

Sin tiempo de expiración: el token vale hasta logout o hasta que se cierre la sesión. La seguridad se apoya en el estado active, el límite de equipos y el cierre remoto.

Caché en el navegador: las pantallas muestran primero una copia y renuevan solas cuando algo cambió. No hay botón de avisos: el refresco automático usa un sello por grupo de datos y respeta el trabajo sin guardar.

Impresión por ticket: el token real nunca viaja en una URL. Se pide un ticket de 60 segundos para iframes y ventanas de impresión.

Auditoría central: casi toda escritura pasa por ActivityLogger. El historial no es un log técnico: es una vista legible para gestión.

Sin colas: QUEUE_CONNECTION=sync. Un envío de backup o de correo bloquea la petición hasta terminar; tenelo en cuenta al dimensionar.

Tamaño del código

Al momento de esta edición, backend/app tiene 94 archivos PHP y unas 16.400 líneas; frontend/src suma 89 archivos JS/JSX (24.200 líneas) y 62 hojas CSS (20.700 líneas). La API expone 103 rutas y el esquema tiene 29 tablas. Son números aproximados: sirven para dimensionar el proyecto, no como métrica de calidad.

Escala objetivo y rendimiento

El sistema se dimensiona para una escuela de más de 1.200 alumnos con uso simultáneo en secretaría, preceptorería y sala de docentes. Ese número no es decorativo: cambió cómo se escriben las consultas. Todo lo que se lee muchas veces (dashboard, historial, asistencia, sello de cambios) se resuelve con agregados SQL, índices y caché, nunca trayendo tablas enteras al PHP.

Padrón completo: se pide paginado y con filtros server-side; la lista de 1.200 alumnos no se carga entera en el navegador y el Excel se arma del lado del servidor, con memoria suficiente y sin reconstruir el DOM de una tabla gigante.

PHP: memory_limit de 256 M o más en producción. Con 128 M, una exportación grande puede cortar la petición.

Sello de cambios: se mantiene en cache por escritura, no con un GROUP BY sobre todo activity_logs; esa versión anterior se volvía insostenible con un año de historial.

Escrituras masivas: la planilla guarda en lote (precarga + upsert) en lugar de una consulta por alumno; con 35 filas por curso y varios cursos por día la diferencia es determinante.

N+1: si una pantalla nueva tarda, contá las consultas. Los endpoints se revisan con route:list + log de queries antes de dar por cerrado un cambio.

Regla de escala

Antes de mergear algo que lea o escriba muchos registros, imaginá 1.200 alumnos y un año de clases: si la solución recorre todo en PHP o dispara una consulta por fila, no pasa la revisión.

Límites conocidos

Una sola institución: no hay multi-tenant. Todos los datos pertenecen a la misma escuela; el aislamiento es por rol y ámbito, no por cliente.

Sin tiempo real: no hay websockets ni push. El refresco automático sondea un sello cada pocos segundos; no es un canal permanente y por eso puede tardar hasta ese sondeo.

Sin colas: backups, correos y exportaciones corren en la misma petición. Una operación pesada puede tardar o cortarse por timeout.

Cobertura parcial: hay pruebas unitarias y de feature, pero no E2E ni pruebas de carga. La verificación manual sigue siendo obligatoria.

Sin control de versiones: el proyecto no es un repositorio git. La única red de seguridad son _backup_*/, backups/ y base-de-datos/, que no son un respaldo automático: hay que generarlos a mano antes de tocar lo delicado.

Archivos fuera de MySQL: la base no contiene los documentos ni los backups. El respaldo de datos debe incluir storage/app.

02 Puesta en marcha

Docker, instalación local y datos iniciales.

La forma recomendada de levantar el proyecto es Docker: reproduce PHP 8.2, MySQL 8, Nginx, Node 22 y Mailpit sin instalar nada global. La instalación local existe como alternativa para depurar con las herramientas del sistema.

Requisitos

Docker Desktop en ejecución y PowerShell.

Puertos libres: 5173 (frontend), 8000 (API), 3307 (MySQL), 1025 y 8025 (Mailpit).

Alternativa local: PHP 8.2 con extensiones pdo_mysql, mbstring, gd, zip; Composer; Node 22; MySQL 8.

Editor con soporte para PHP, JavaScript/JSX y SQL. No hace falta un IDE específico.

Arranque con Docker

```
# Desde la raíz del repositorio
.\scripts\iniciar.ps1

# Equivalente manual
docker compose build
docker compose up -d
docker compose exec backend sh -c "[ ! -d vendor ] && composer install || echo 'vendor ya existe'"
docker compose exec backend php artisan migrate --force
```

Frontend: http://localhost:5173

API: http://localhost:8000/api/ping debe devolver {"ok":true}

Correos: http://localhost:8025 (Mailpit; no envía a internet)

MySQL: localhost:3307, usuario root, clave root, base sistema_escuela.

Variables de entorno

En Docker, backend/.env.docker es la configuración oficial y docker-compose.yml pisa las DB_* de forma explícita. En local, partí de backend/.env.example. Nunca subas un .env con credenciales reales al repositorio.

Variable	Para qué sirve
APP_ENV / APP_DEBUG	local con debug; producción con debug apagado.
APP_KEY	clave de cifrado de Laravel. No se comparte ni se versiona.
APP_TIMEZONE	America/Argentina/Buenos_Aires. Si se pone UTC, las fechas del historial y la asistencia corren tres horas.
DB_HOST / PORT / DATABASE	conexión MySQL. En Docker el host es mysql:3306 y el compose pisa estos valores.
ENRO_DB_PASSWORD / ENRO_HTTP_PORT	variables del docker-compose: la clave de MySQL y el puerto público del nginx (8000 por defecto).
FRONTEND_URL	origen permitido por CORS y base del enlace de recuperación de contraseña.
ADMIN_EMAIL / ADMIN_PASSWORD	admin inicial del seeder. Si la clave está vacía, se genera una y se muestra una vez en la salida.
PLANILLA_*	encabezado, nombre de la escuela y título que salen impresos en la planilla oficial y en el padrón de Excel.
MAIL_*	SMTP. En Docker apunta a mailpit:1025; en producción, un servidor real.
CACHE_DRIVER / SESSION_DRIVER	file en la configuración actual; los tests usan array.
DB_BACKUP_MYSQLDUMP	ruta al mysqldump cuando no está en el PATH; si falta, el backup cae a modo JSON.
SLOW_REQUEST_MS	umbral del middleware que deja en el log las peticiones lentas; 2000 por defecto.
VITE_API_BASE	URL de la API que consume el frontend. Se quema en el bundle al compilar: cambiarla exige reconstruir.

Datos iniciales

```
docker compose exec backend php artisan db:seed --force

# Si partís de una base vacía y querés el set completo:
docker compose exec backend php artisan migrate --seed --force
```

Cuidado con el seed: artisan db:seed siempre corre AdminUserSeeder, ParametroDivisionesSeeder, CursosSeeder y, además, AlumnoSeeder y DemoDataSeeder, que cargan divisiones, cursos, materias, alumnos de muestra, horarios y calificaciones de ejemplo. La clave del admin sale de ADMIN_PASSWORD; si está vacía, el seeder genera una aleatoria y la muestra una sola vez en la salida. No existe una variable que active o desactive los datos de muestra: en una escuela real se siembra, se prueba la pantalla y después se borran esos alumnos y esas notas, o se cargan los datos sin correr DemoDataSeeder.

Sembrar sobre datos reales es una mala idea
Los seeders de demostración no son idempotentes: volver a correrlos contra una base con alumnos de verdad choca contra el DNI único y deja filas duplicadas donde no las había. Antes de sembrar, confirmá en qué base estás parado: php artisan tinker y después DB::connection()->getDatabaseName().

Instalación local sin Docker

```
cd backend
composer install
copy .env.example .env
php artisan key:generate
# Editar .env: DB_HOST=127.0.0.1, DB_PORT=3306, DB_DATABASE, DB_USERNAME, DB_PASSWORD
php artisan migrate --seed
php artisan storage:link
php artisan serve

# En otra terminal
cd frontend
npm install
npm run dev
```

En local, el frontend usa VITE_API_BASE o cae en http://localhost:8000/api. Si cambiás el puerto de Laravel, definí la variable antes de arrancar Vite.

Desarrollo en Windows con Laragon

El entorno de desarrollo de este proyecto es Laragon sobre Windows, con la base MySQL sistema_escuela. No hay proxy entre Vite y Laravel: el frontend llama directo a http://localhost:8000/api. Los binarios de PHP y Node no están en el PATH del sistema, así que en scripts y en la terminal se usan las rutas completas de Laragon.

```
# PHP (ajustá la versión a la que tengas instalada)
/c/laragon/bin/php/php-8.3.33-Win32-vs16-x64/php.exe artisan migrate
/c/laragon/bin/php/php-8.3.33-Win32-vs16-x64/php.exe artisan route:list
/c/laragon/bin/php/php-8.3.33-Win32-vs16-x64/php.exe artisan tinker archivo.php

# Node / ESLint / Vite, sin npm start
/c/laragon/bin/nodejs/node-v22/node.exe node_modules/eslint/bin/eslint.js src --quiet
/c/laragon/bin/nodejs/node-v22/node.exe node_modules/vite/bin/vite.js build
```

API en http://localhost:8000, frontend en http://localhost:5173: los dos corren por separado, sin proxy.

Verificación obligatoria de un lote: php -l en cada archivo tocado, eslint --quiet sin errores nuevos, vite build sin fallos y una prueba real en el navegador.

Harnesses con tinker: las pruebas de modelo se escriben en un archivo PHP temporario y se borran después; cualquier escritura a la base de desarrollo va dentro de DB::beginTransaction() con rollBack, salvo limpieza propia verificada.

Producción es Linux, no Laragon

Lo que se prueba en Windows no es lo que se despliega. El camino real es Docker con Nginx + PHP-FPM sobre Linux (ver docs/DESPLEGUE-LINUX.md). Nada que dependa de rutas de Laragon, de la letra de una unidad o de las mayúsculas de Windows puede quedar en el código que se sube.

Nada bloquea un migrate:fresh: el bloqueo sos vos

El proyecto no tiene una variable que habilite o impida los comandos destructivos; artisan los corre siempre. migrate:fresh, migrate:refresh, migrate:reset y db:wipe borran la base conectada sin pedir confirmación cuando van con --force. Antes de escribir uno, verificá el destino con php artisan tinker y DB::connection()->getDatabaseName(), y confirmá que exista un backup probado. Si dudás, no ejecutes: pedí el backup y confirmá con otra persona.

Comandos diarios

Comando	Para qué
docker compose logs -f backend nginx	Ver errores de Laravel y del servidor web.
docker compose exec backend php artisan about	Resumen de entorno y configuración.
docker compose exec backend php artisan route:list	Ver rutas, middleware y controladores.
docker compose exec backend php artisan tinker	Consultas puntuales y pruebas de modelos.
docker compose exec mysql mysql -uroot -proot	Entrar a MySQL (solo diagnóstico).
docker compose down	Detener sin borrar el volumen de datos.
docker compose down -v	Detener y borrar la base. Solo si estás seguro.

03 Estructura del repositorio

Dónde vive cada cosa y cómo nombrarla.

El repositorio está organizado por proceso, no por tipo de archivo. Si vas a tocar asistencia, buscás Asistencia en el backend y asistencia en el frontend. Esa simetría es la convención más importante del proyecto.

Árbol de primer nivel

```
Sistema-ENRO-main/
├─ backend/              API Laravel (código activo)
│   ├─ app/Dominios/     un proceso por carpeta
│   ├─ app/Support/      reglas y ayudas compartidas
│   ├─ app/Http/         middleware y kernel
│   ├─ routes/           api.php y web.php
│   ├─ database/         migraciones y seeders
│   └─ tests/            PHPUnit
├─ frontend/             SPA React (código activo)
│   └─ src/
│      ├─ modulos/       pantallas por dominio, cada .jsx con su .css al lado
│      └─ compartido/    api, componentes, hooks, lib y styles (solo el global)
├─ archivos/             plantillas de Excel que lee el backend
├─ base-de-datos/        esquema y respaldos históricos (solo lectura)
├─ backups/              respaldos que genera el script
├─ docker/               Dockerfiles y Nginx
├─ docs/                 instalación, mapa y manuales
├─ scripts/              iniciar / detener
└─ _backup_*/            estado previo a los arreglos (no editar)
```

Dónde tocar y dónde no

Una pantalla se busca en frontend/src/modulos/<dominio>/ y su lógica en backend/app/Dominios/<Dominio>/: los nombres están espejados. El mapa proceso → archivo → endpoint → tabla está en docs/MAPA_SISTEMA.md y el listado completo de rutas en backend/routes/api.php. El proyecto no usa git, así que la única red de seguridad son las carpetas _backup_*, backups/ y base-de-datos/: no se editan ni se borran sin confirmación del equipo.

Backend

Dominios: Alumnos (9 archivos), Asistencia (4), Calificaciones (9), Cursos (11), Documentos (2), Profesores (6), Sistema (5), Usuarios (6).

Support: ActivityDiff, ActivityLogger, AsistenciaReglas, AuthorizesScope, CourseLabel, Descarga, LikeSearch, MateriaResolver, Paginacion, PadronExcel, PlanillaExcel, RoleHierarchy, Rules/EmailInstitucional, SchoolYear, SlotKey y Suplencias. El sello de cambios vive en Dominios/Sistema/SelloCambios.

Http: Kernel, middleware AuthToken, EnsureRole, LogSlowRequests y los middleware estándar de Laravel.

Rutas: api.php concentra la API; web.php contiene planillas, Excel y listados imprimibles.

Base: database/migrations y database/seeders. No edites una migración ya aplicada en producción.

Frontend

Módulos: alumnos, asistencia, auth, calificaciones, cursos, dashboard, documentos, historial, preceptores, profesores y usuarios.

Compartido: lib (api, apiGate, authToken, pageCache, dataStamp, imprimir, descarga, divisiones, cursos, asistencia, busqueda, fechas, orden, errores, export y deshacer), componentes (layout, modals, Common), hooks (useAvisoDeCambios, useConfirmAction) y styles, donde solo quedan variables.css y base.css: el CSS de cada pantalla vive en su carpeta de modulos/, al lado del .jsx que lo usa.

Convenciones de nombre

Tipo	Convención
Controlador	PascalCase + Controller, dentro del dominio. Ej.: AlumnoController.
Modelo	PascalCase, singular, en el dominio. La tabla va en snake_case plural.

Tipo	Convención
Migración	timestamp + create_/add_ + tabla. No se renombra después de aplicar.
Ruta API	sustantivo en plural: /alumnos, /calificaciones, /horarios.
Componente React	PascalCase.jsx; hooks useAlgo.js; helpers en lib/.
Hoja de estilos	Mismo nombre y carpeta que su pantalla (modulos/<dominio>/<Pantalla>.css). Solo lo global va en compartido/styles/.
Texto visible	español rioplatense, sin jerga técnica para el usuario final.

04 Backend Laravel

Rutas, middleware, controladores, modelos y soporte.

El backend es una aplicación Laravel 10 clásica, con una particularidad: los modelos y controladores no viven en app/Models y app/Http/Controllers, sino en app/Dominios, agrupados por proceso. Esa decisión evita un directorio plano gigante y hace evidente qué archivos se tocan juntos.

Ciclo de una petición

1. Nginx recibe la URL y entrega public/index.php a PHP-FPM.
2. Laravel carga el Kernel, el grupo de middleware global y el proveedor de rutas.
3. La ruta se resuelve en api.php o web.php y se aplican los middleware declarados.
4. AuthToken resuelve el usuario; EnsureRole aplica la jerarquía; el controlador valida el ámbito.
5. El controlador valida, ejecuta la operación, registra la actividad y devuelve JSON o una vista.
6. Las excepciones se transforman en respuestas 401, 403, 404, 422 o 500 con un campo message.

Rutas y grupos

api.php se divide en rutas públicas y un gran grupo protegido con auth.token. Dentro de ese grupo hay subgrupos por rol. Leé los comentarios del archivo: explican por qué una ruta está en un grupo y no en otro.

```
Route::middleware('auth.token')->group(function () {
    Route::get('/alumnos', [AlumnoController::class, 'index']);

    Route::middleware('role:admin,directivo,jefe_preceptores')->group(function () {
        Route::post('/alumnos', [AlumnoController::class, 'store']);
        Route::post('/alumnos/promocionar/preview', [PromotionController::class, 'preview']);
    });

    Route::middleware('role:admin,directivo,jefe_preceptores,preceptor')
        ->group(function () {
            Route::post('/documentos', [DocumentoController::class, 'store']);
        });
});
```

Públicas: ping, login, register, forgot y reset. Tienen throttle propio.

Lectura autenticada: alumnos, calificaciones, profesores, cursos, horarios, dashboard y documentos.

Gestión: admin, directivo y jefe_preceptores para altas, bajas, promoción, ciclo y backups.

Ámbito: preceptor y prosecretaria escriben solo en sus cursos; profesor solo califica en su ámbito.

Alias: hay rutas duplicadas por compatibilidad (/cursos/{anio}/{division}/horario y /horarios/...). No agregues una tercera variante.

Middleware

Middleware	Qué hace
auth.token	Lee Bearer o ?ticket=. Busca en api_sesiones y usa users.api_token como respaldo. Exige status active.
role:...	EnsureRole compara niveles: admin 5, directivo 4, jefe 3, preceptor/prosecretaria 2, profesor 1.
throttle:api	60 requests por minuto por usuario o IP.
throttle:login	5 intentos por minuto por email + IP, no por IP sola.
throttle:password	5 intentos cada 10 minutos para recuperación de contraseña.

role es jerárquico, no una lista exacta

role:admin,jefe_preceptores deja pasar a admin, directivo y jefe_preceptores, porque directivo tiene nivel 4 y el mínimo pedido es 3. Si necesitás una lista exacta, no uses role: validá el rol dentro del controlador. El test EnsureRoleTest protege este comportamiento.

Controladores

Validación: casi siempre con $request->validate([...]) dentro del método. Los mensajes salen en español; el frontend muestra data.message.

Autorización: el trait AuthorizesScope aporta userCanWriteAll, userCanGrade, userIsAssignedTo, assertCanWriteAlumno y assertCanGradeAlumno.

Respuestas: JSON con claves descriptivas; los listados usan paginación de Laravel; los errores usan abort() o response()->json([...], código).

Transacciones: se usan en promoción, ciclo, calificaciones y operaciones que tocan varias tablas. Si agregás una escritura compuesta, usá DB::transaction.

Auditoría: después de una escritura relevante, llamá a ActivityLogger::log con una acción legible y un action_type estable.

Modelos y dominio

Cada modelo declara table y fillable. Las relaciones son explícitas y los accesores resuelven etiquetas visibles (por ejemplo, el curso "4to B"). No hay un modelo universal: si necesitás algo de alumnos, buscá en Alumnos; si es de notas, en Calificaciones.

```
namespace App\Dominios\Alumnos;

class Alumno extends Model
{
    protected $table = 'alumnos';
    protected $fillable = ['nombre', 'apellido', 'dni', 'anio', 'division', ...];

    public function responsables() {
        return $this->hasMany(AlumnoResponsable::class);
    }
}
```

Alumno: datos personales, curso, estado, banderas PPI/AE, especialidad y datos de egreso.

Curso / Materia / HorarioCelda: estructura académica y grilla semanal.

Clase / Asistencia / Justificación: registro diario y cadena de justificaciones.

AlumnoCalificacion / AlumnoPromedio / AlumnoMateriaEstado: notas, promedios persistidos y estado de materia.

User / ApiSesion: cuenta, rol, estado y equipos conectados.

Clases de soporte

Clase	Responsabilidad
AuthorizesScope	Permisos por rol y ámbito; es el corazón de la seguridad fina.
RoleHierarchy	Niveles y regla canManage para gestionar usuarios.
AsistenciaReglas	Estados, etiquetas y fórmulas de porcentaje.
SchoolYear	Año máximo desde ParametroDivision, con fallback 7.
CourseLabel	Etiquetas ordinales ("4to 2da") en un solo lugar.
Paginacion	Normaliza per_page entre 1 y 200; nunca rompe el SQL.
PlanillaExcel	Diseño oficial de calificaciones: 35 filas por 24 columnas con PhpSpreadsheet.
PadronExcel	Padrón de alumnos en .xlsx: hoja PADRÓN (23 columnas, autofiltro) y hoja RESUMEN.
Descarga	Headers Content-Disposition de .xlsx y .csv: nombre ASCII y filename* en UTF-8, para que no salgan "?" en Windows.
MateriaResolver	Desempata materias homónimas de un mismo año con la especialidad del curso (Dibujo Técnico existe en Electromecánica y en MMO).
SelloCambios	Número por grupo de datos que alimenta el refresco automático; se mantiene en cache por escritura.
SlotKey	Normaliza claves horarias h1..h9.
Suplencias	Valida cadenas, ciclos y profundidad máxima de 4.
ActivityLogger	Registra la entrada de historial con usuario y diff.

Clase	Responsabilidad
ActivityDiff	Reduce old/new a lo que realmente cambió.

Auditoría y consistencia

ActivityLogger no guarda el request completo: guarda una acción legible, un action_type estable para agrupar, el tipo/id afectado y un diff. Si old y new quedan vacíos, no genera entrada (una edición sin cambios no ensucia el historial). Las altas y bajas sí se registran siempre.

```
ActivityLogger::log(
    $request,
    'Editó el alumno ' . $alumno->apellido,
    'alumno.update',
    'Alumno',
    $alumno->id,
    $antes,
    $despues
);
```

Diff: ActivityDiff compara y descarta campos intactos. Los formatos especializados (materias, planilla, asignaciones) se conservan completos.

Motivo: si el request trae _detalles, se guarda como motivo de la acción.

Caché: el historial se cachea 300 segundos sin filtros; cualquier entrada nueva debe invalidarlo.

Errores de unicidad: las restricciones de MySQL son la última barrera. Atrapá QueryException solo si podés dar un mensaje útil.

05 Base de datos

Tablas, relaciones, índices y migraciones.

MySQL 8 es la fuente de verdad. El esquema se crea con migraciones y se completa con seeders. La conexión usa utf8mb4_unicode_ci y strict mode: si un dato no entra, la operación falla en vez de guardarse truncada.

Grupos de tablas

Grupo	Tablas
Personas	users, alumnos, profesores, profesor_asignaciones, profesor_suplencias, profesor_bajas, user_anio_division.
Académico	cursos, materias, horario_celdas, parametro_divisiones, alumno_horario_overrides.
Calificaciones	alumno_calificaciones, alumno_promedios, alumno_materias_estado, alumno_intensificaciones, alumno_recursandos, ventana_nota_final.
Asistencia	clases, asistencias, asistencia_justificaciones.
Documentos	documentos (archivo en storage/app/documentos).
Sistema	activity_logs, api_sesiones, password_reset_tokens.
Datos de alumno	alumno_direcciones, alumno_telefonos, alumno_responsables.

Tablas críticas

Tabla	Clave / restricción	Para qué
clases	UNIQUE(curso_id, fecha); UNIQUE(curso_id, numero_clase)	Una toma de asistencia por curso y día.
asistencias	UNIQUE(clase_id, alumno_id); FK a clases y alumnos	Estado individual; impide doble carga.
horario_celdas	UNIQUE(curso_id, dia, slot_key, grupo)	Una materia/profesor por celda y grupo.
alumnos	dni UNIQUE; índices en (anio, division), (apellido, nombre), estado, especialidad y matricula	Padrón, búsquedas y el Excel de la pantalla Alumnos.
alumno_calificaciones	FK alumno/materia; periodo y tipo	Notas regulares, preliminares y finales.
api_sesiones	token único; user_id	Una sesión por equipo, máximo 10.
activity_logs	action_type indexado; auditable	Historial y sello de cambios.
ventana_nota_final	fila única lógica (la última)	Fechas y alcance p1/p2/ambos.

Relaciones principales

Alumno: hasMany direcciones, teléfonos, responsables, recursandos e intensificaciones. Pertenece a un curso lógico por anio + division.

User: hasMany user_anio_division y api_sesiones. El rol y el estado definen el acceso.

Curso: tiene materias por año, celdas de horario y clases. No guarda notas directamente.

Clase: pertenece a Curso y User (responsable); hasMany asistencias.

Asistencia: pertenece a Clase y Alumno; unique por ambos.

Calificación: pertenece a Alumno y Materia; se agrupa por ciclo, año, división y periodo.

HorarioCelda: pertenece a Curso, Materia y Profesor; se identifica por día, slot y grupo.

Reglas de esquema

No edites una migración ya aplicada en producción: creá una nueva con timestamp. Así entraron nacionalidad y matricula a alumnos, con su índice en matricula.

Toda FK nueva debe declarar onDelete/onUpdate explícito. El esquema actual usa cascade en asistencias y clases.

Los índices no son decoración: `clases(curso_id, fecha)` y `asistencias(clase_id, estado)` sostienen las consultas diarias.

Un seeder que se puede correr dos veces sin duplicar nada usa firstOrCreate/updateOrCreate. CursosSeeder y ParametroDivisionesSeeder lo hacen; AlumnoSeeder y DemoDataSeeder no, y por eso sólo corren en una base vacía.

No guardes archivos binarios en MySQL. Los documentos van a storage/app/documentos y la tabla guarda la ruta.

Antes de una migración destructiva, backup verificado y ventana de mantenimiento.

El reset no es una herramienta de desarrollo diaria

migrate:fresh borra todas las tablas y no pide confirmación si el comando se corre con --force. En una base con datos reales no hay segunda oportunidad. Si necesitás reproducir un bug, creá una base aparte, corré las migraciones ahí y cargá datos ficticios; nunca uses la base de la escuela como entorno de prueba.

Migraciones y seeders

```
# Ver estado sin ejecutar nada
docker compose exec backend php artisan migrate:status

# Aplicar solo migraciones nuevas
docker compose exec backend php artisan migrate --force

# Set base + datos de demostración (solo en una base vacía)
docker compose exec backend php artisan db:seed --force
```

Seeders del proyecto: AdminUserSeeder, ParametroDivisionesSeeder, CursosSeeder, AlumnoSeeder, DemoDataSeeder, ProfesorSeeder y DatabaseSeeder, que los encadena en ese orden. Ojo con el orden y con lo que carga: DatabaseSeeder siempre trae los datos de muestra de AlumnoSeeder y DemoDataSeeder (materias, alumnos, horarios y notas de ejemplo), no hay bandera que los saltee. ProfesorSeeder corre al final porque necesita las materias que crea DemoDataSeeder.

06 API, sesiones y permisos

Token propio, límites, errores e impresión.

La API es JSON y stateless desde el punto de vista del navegador: no usa cookies de sesión para autenticar. El frontend guarda un token y lo envía en cada request. Esa decisión simplifica el despliegue, pero exige cuidar el almacenamiento local y el cierre de sesiones.

Flujo de login

1. POST /api/login con email y password. Throttle: 5 por minuto por email + IP.
2. Laravel verifica Hash::check y que status sea active.
3. Se eliminan las sesiones más antiguas si ya hay 10; se abre una nueva en api_sesiones.
4. La respuesta incluye user y token. El frontend guarda el token en localStorage.
5. Cada request envía Authorization: Bearer <token>. AuthToken resuelve el usuario y exige active.
6. POST /api/logout cierra la sesión actual; con todos=true cierra todas.

```
# Login
POST /api/login
{ "email": "usuario@abc.gob.ar", "password": "..." }

# Respuesta
{ "user": { "id": 1, "role": "admin", ... }, "token": "..." }

# Request autenticado
GET /api/me
Authorization: Bearer <token>
```

Características del token

Almacenamiento: api_sesiones guarda una fila por equipo, con token, dispositivo, IP, último uso y user_id.

Respaldo: si el token no está en api_sesiones, AuthToken busca users.api_token. Es compatibilidad con sesiones viejas, no el camino principal.

Caché: el usuario se cachea 5 minutos por token. Un cambio de rol puede tardar hasta ese plazo en reflejarse.

Límite: máximo 10 equipos. Al superarlo se eliminan las sesiones más antiguas.

Expiración: no hay expiración por tiempo. La sesión termina por logout, cierre remoto o status distinto de active.

URLs: el token real nunca va en query string. Para imprimir se usa un ticket de 60 segundos.

> **401 no siempre significa "token vencido"**
> También puede ser un token borrado, un usuario desactivado o un header mal formado. Antes de tocar código, revisá localStorage, /api/me y el estado del usuario. El frontend limpia la sesión y redirige al login ante cualquier 401.

Límites de frecuencia

Límite	Cuota	Alcance
api	60/min	Por usuario autenticado o IP.
login	5/min	Por email + IP, para no bloquear a toda la escuela.
password	5/10 min	Recuperación y restablecimiento.
register	10/10 min	Por IP; las cuentas quedan pendientes.
backups	20/60 min	Operaciones pesadas de backup.

Mapa de endpoints

Grupo	Rutas principales
Sesión	POST /login, /logout; GET /me, /me/ambito; PUT /profile; POST /print/ticket.
Alumnos	GET/POST /alumnos; GET /alumnos/padron/excel (baja el .xlsx del padrón); GET/PUT/DELETE /alumnos/{id}; POST /alumnos/{id}/egresar; promoción preview/aplicar.
Asistencia	GET/POST /asistencia/clases; GET/PUT/DELETE /asistencia/clases/{id}; historial, inasistencias, justificaciones, estadísticas.
Calificaciones	GET/PUT /alumnos/{id}/calificaciones; PUT /calificaciones/batch; resumen; por-curso-materia; ventana.
Cursos	GET/POST /cursos; PATCH/DELETE /cursos/{id}; GET /divisiones; POST /cursos/sync; GET/PUT /horarios/{anio}/{division}.
Profesores	GET/POST /profesores; PUT /profesores/{id}; suplencias; mapping; from-users; force delete.
Documentos	GET/POST /documentos; PUT/DELETE /documentos/{id}; GET download/print.
Sistema	GET /historial; POST /historial/refresh; GET /datos/sello; dashboard; backups.
Impresión	En web.php: GET /calificaciones/planilla; /planilla/excel; POST /planilla/guardar; /horarios/planilla; /print/asistencia. El padrón de alumnos se baja por la API (GET /alumnos/padron/excel); /print/alumnos quedó sin uso desde la pantalla.

Las vistas imprimibles (HTML y PDF) viven en web.php y las descargas de datos (JSON, el .xlsx del padrón) en api.php. Las dos usan el mismo middleware auth.token; la diferencia es qué devuelven.

Paginación y errores

Listados: paginación de Laravel. Podés pasar per_page; Paginacion lo limita entre 1 y 200, con default 50.

Búsqueda: LikeSearch::contains arma el LIKE con escape para no romper comodines.

401: falta token, token inválido o usuario inactivo. El frontend cierra sesión.

403: rol suficiente pero sin ámbito, o rol insuficiente para la ruta.

404: recurso inexistente; algunos controladores devuelven un mensaje en español.

422: validación o regla de negocio (por ejemplo, ventana cerrada o contraseña no confirmada).

429: throttle. Esperá la ventana indicada; no reintentes en loop.

500: error no controlado. Mirá storage/logs/laravel.log y el stack trace.

Impresión y archivos

Las planillas y listados no se generan en React: se generan en Blade/DomPDF o PhpSpreadsheet y se abren en un iframe. Para que el iframe pueda autenticarse sin exponer el token, el frontend pide un ticket y lo agrega a la URL.

```
// 1) ticket de 60 s
POST /api/print/ticket  ->  { "ticket": "..." }

// 2) URL imprimible (sin token real)
GET /calificaciones/planilla?anio=4&division=2&materia_id=12&ticket=...

// 3) el iframe llama print(); el backend acepta el ticket
// durante 60 s y lo resuelve contra el usuario que lo pidió.
```

PDF: DomPDF renderiza la vista Blade. Si una tabla crece, revisá el CSS de impresión y el tamaño de página.

Excel: PhpSpreadsheet arma el .xlsx real; no es una captura. Hay dos constructores: PlanillaExcel (planilla oficial) y PadronExcel (padrón de alumnos). Requiere las extensiones zip y gd, y memory_limit amplio para padrones grandes.

Nombres de archivo: Descarga::xlsx() pone el nombre dos veces, ASCII y UTF-8, y suma Cache-Control: no-store. Si un nombre lleva acentos o la ñ, no lo armes a mano en el controlador.

Descargas: apiBlob devuelve un Blob; el token va en headers, no en la URL.

Subidas: apiUpload usa multipart/form-data; no setees Content-Type a mano (el navegador pone el boundary).

> **No agregues endpoints de impresión a api.php sin pensar**
> Las rutas web devuelven HTML/PDF/XLS, no JSON. Si las movés al grupo api, el cliente y el manejo de errores cambian. Mantené la separación: API para datos, web para artefactos imprimibles.

07 Frontend React

Rutas, guards, cliente API, caché y refresco automático.

El frontend es una SPA con React 19 y Vite 6. Cada módulo se carga con lazy() para no descargar toda la aplicación al inicio. La lógica de red está centralizada en un solo archivo; si una pantalla hace fetch por su cuenta, está fuera de la convención.

Entrada y rutas

main.jsx monta BrowserRouter, AuthProvider y App. App.jsx define las rutas públicas, las protegidas y el layout con Navbar condicional. Las rutas desconocidas redirigen a /dashboard.

Ruta	Acceso
/login, /register	Solo invitados (RequireGuest).
/recuperar, /restablecer	Recupero de contraseña.
/dashboard	Autenticado.
/alumnos, /cursos, /profesores	Autenticado; el backend filtra el ámbito.
/preceptores	admin, directivo, jefe, preceptor, prosecretaria.
/jefe-preceptores	admin, directivo, jefe_preceptores.
/calificaciones, /asistencia	Autenticado; el backend valida el ámbito.
/admin/usuarios, /historial	admin, directivo, jefe_preceptores.
/documentos	Autenticado; visibilidad por rol en el backend.

Autenticación en el cliente

AuthProvider: restaura la sesión con /me si hay token; guarda user, ready, login, logout y updateProfile.

Token: se guarda en localStorage mediante authToken.js. No se guarda la contraseña.

401 global: api.js llama a handleUnauthorized, limpia el token y emite auth:expired; AuthProvider pone user en null.

Logout: limpia token, caché y sellos. Con todos=true cierra también los otros equipos.

Guards: RequireAuth conserva la URL de destino en ?to=; RequireRole redirige al inicio si el rol no alcanza.

El cliente API

Todas las llamadas pasan por frontend/src/compartido/lib/api.js. El archivo agrega headers, interpreta errores, maneja 401, invalida caché después de una escritura y marca el cambio propio para que el refresco automático no pise la pantalla que acaba de guardar.

```
import { apiGet, apiPost, apiBlob } from "../../compartido/lib/api";

// Lectura con señal de cancelación
const data = await apiGet(`/alumnos?search=${q}`, { signal });

// Escritura: api.js invalida las copias relacionadas
await apiPost("/alumnos", { nombre, apellido, anio, division });

// Archivo binario (documentos, Excel del padrón)
const blob = await apiBlob(`/alumnos/padron/excel?${params}`);
descargarBlob(blob, `lista_alumnos-${fecha}.xlsx`);
```

apiGet / apiPost / apiPut / apiDelete: JSON. Lanzan Error con status y data; el mensaje ya viene en español.

apiUpload: multipart para documentos. No setees Content-Type manualmente.

apiBlob: descargas binarias (PDF, .xlsx). El token va en headers, nunca en la URL; después se crea un object URL y se descarga con descargarBlob().

AbortSignal: todas aceptan un signal suelto o { signal } para cancelar al desmontar.

Caché de pantallas

pageCache.js mantiene una copia en memoria y, si es chica (menos de ~120 KB), también en localStorage. La versión de esquema es 4: si cambia la forma de los datos, incrementala para invalidar copias viejas.

Memoria: respuesta instantánea al navegar dentro de la SPA.

LocalStorage: arranque en frío; se descarta si supera el límite.

Invalidación: api.js tiene un mapa ruta → claves. Si agregás una escritura nueva, revisá si necesita invalidar algo.

Entre pestañas: activarSincronizacionEntrePestanas escucha el evento storage y descarta la copia en memoria.

Historial: se invalida en cualquier escritura porque casi todo genera una entrada.

Refresco automático (sello de cambios)

El backend expone GET /api/datos/sello: un número por grupo (alumnos, profesores, cursos, asistencia, calificaciones, usuarios, documentos, otros). Ese número es el id del último movimiento de activity_logs del grupo. El frontend lo compara con el que anotó la pantalla y, si subió, vuelve a pedir solo los datos de ESA pantalla: sin recargar la SPA, sin perder filtros y sin volver arriba de la lista. No hay botón ni aviso que pulsar.

```
// La pantalla declara su nombre y los grupos que lee.
useAvisoDeCambios("alumnos", ["alumnos", "cursos"], actualizarTodo);

// No devuelve nada: actualizarTodo se ejecuta sola cuando
// aparece un cambio ajeno y la pantalla está libre.
```

Sondeo: cada 12 segundos con la pestaña visible; también al volver a la pestaña o enfocar la ventana, con una marca que deduplica los dos eventos para no consultar dos veces.

Pantalla ocupada: no se refresca si hay un modal abierto (role="dialog" o aria-modal), un campo con el foco, marcas de asistencia sin guardar (el atributo data-cambios-sin-guardar en el body) o un cambio propio vigente.

En espera: el cambio ajeno no se pierde: queda marcado y entra 350 ms después de cerrar el modal o salir del campo, o en el próximo sondeo.

Cambio propio: api.js llama a marcarCambioPropio() tras una escritura; durante 20 segundos esa pantalla ignora el sello, porque lo que ve ya es lo último.

Backend: SelloCambios actualiza sus claves en cada ActivityLogger::log, así que leer el sello son lecturas de cache y no un GROUP BY sobre todo el historial. Las claves son sello:grupo:(una por grupo) y el driver es file.

Grupos: el prefijo del action_type decide el grupo (alumno., calificaciones, profesor., asistencia., curso., documento., user....). Lo desconocido cae en "otros"; user.login y user.logout se ignoran a propósito.

Cierre de sesión: borrarSellos() limpia lo guardado en el navegador para que la próxima cuenta no herede sellos ajenos.

> **No vuelvas a poner un botón de novedades**
> El refresco es silencioso por decisión del responsable funcional del sistema: con varias terminales abiertas, un botón que parpadea se convertía en trabajo extra. Si una pantalla nueva necesita datos al día, declará sus grupos en useAvisoDeCambios(); no crees un aviso manual ni un setInterval paralelo.

Impresión y descargas

pedirTicketImpresion: POST /print/ticket; devuelve un ticket de 60 segundos.

imprimirUrl: crea un iframe oculto, espera onload y llama print(). No usa window.open, así que los bloqueadores de popups no interfieren.

PDF/Excel: se piden con apiBlob y se descargan con un object URL; el token va en headers.

Planilla: la vista Blade y el Excel comparten el mismo diseño; si cambiás columnas, actualizá ambos.

Componentes y UX

Layout: Navbar, Breadcrumbs y FichaLayout. El menú se arma según el rol.

Estados: EmptyState para listas vacías, Toast para confirmaciones, OfflineBanner para falta de conexión.

Modales: AboutModal, AccountModal y modales de confirmación de acciones destructivas.

Gráficos: recharts en Dashboard y estadísticas de asistencia; lucide-react para los íconos.

Deshacer: lib/deshacer.jsx ofrece el botón "Deshacer" dentro de la notificación de éxito unos segundos; los flujos delicados piden contraseña (exigePassword). No reemplaza la transacción del backend ni borra el registro del Historial.

Animación: framer-motion solo para mover componentes; la aparición de los modales se resuelve con keyframes CSS, porque la opacidad animada por Framer provocaba parpadeos y pérdida de foco.

Responsive: el menú se vuelve hamburguesa en celular, las tablas scrollean en horizontal en vez de aplastarse y las alturas usan dvh en lugar de 100vh. Verificá 360, 768 y 1280 px; la planilla y las tablas largas siguen priorizando escritorio.

No dupliques la capa de red

Si una pantalla necesita un endpoint nuevo, agregá la función en api.js o en el helper del módulo, pero no armes un fetch con token a mano. El manejo de 401, errores, caché e invalidación se rompe en cuanto hay dos caminos.

08 Módulos críticos

Qué toca cada proceso y dónde está el riesgo.

Esta sección no reemplaza la lectura del código: señala los archivos y las reglas que hay que mirar antes de modificar cada módulo. Si vas a hacer un cambio, empezá por la tabla y el controlador; después seguí por el frontend.

Mapa rápido

Módulo	Archivos clave	Tablas / riesgo
Alumnos	Dominios/Alumnos; modulos/alumnos	alumnos y anexos; altas, egreso y promoción.
Asistencia	Dominios/Asistencia; modulos/asistencia	clases, asistencias, justificaciones; unique por día.
Calificaciones	Dominios/Calificaciones; modulos/calificaciones	notas, promedios, estados, planilla; reglas normativas.
Cursos	Dominios/Cursos; modulos/cursos	cursos, materias, horarios, divisiones; año tope.
Profesores	Dominios/Profesores; modulos/profesores	asignaciones y suplencias; ámbito.
Documentos	Dominios/Documentos; modulos/documentos	archivos y visibilidad por rol; storage.
Usuarios	Dominios/Usuarios; modulos/usuarios	roles, estado, sesiones; no tocar superiores.
Sistema	Dominios/Sistema; modulos/historial	historial, sello, dashboard, backups.

Alumnos

Listado: búsqueda por nombre, apellido, DNI o matrícula y filtros por año, división, género, especialidad y orden. El backend aplica scopeAlumnoIndex: si el usuario no tiene cursos asignados, devuelve una lista vacía, no todos.

Ficha: datos personales —incluidos nacionalidad y N° de matrícula—, contacto, responsables, dirección, observaciones, PPI/AE, especialidad y datos de egreso. La ficha completa se reserva a admin/directivo/jefe.

Alta/edición: admin/directivo/jefe en cualquier curso; preceptor/prosecretaria solo en su ámbito.

Egreso: POST /alumnos/{id}/egresar guarda egreso_year, egreso_anio y egreso_division; no borra la historia.

Promoción: preview y aplicar; mueve años masivamente. Requiere confirmación de contraseña y se ejecuta en transacción.

Padrón en Excel: GET /alumnos/padron/excel (AlumnoController::exportarPadron, alrededor de la línea 805) arma las filas, las pasa por PadronExcel y responde con Descarga::xlsx(). El nombre del archivo sale con la fecha de la descarga: lista_alumnos-DD-MM-AAAA.xlsx.

Cuidado con el padrón: las filas se preparan como texto/simple en el controlador; PadronExcel sólo formatea. Si agregás una columna, tocá COLUMNAS del PadronExcel, el arreglo de filas del controlador y la hoja RESUMEN si el dato cambia los conteos.

Asistencia

Modelo: una clase es una toma de asistencia de un curso en una fecha. UNIQUE(curso_id, fecha) impide dos tomas el mismo día.

Estados: presente, ausente, justificado y tarde. Una clase puede ser dictada o suspendida.

Justificaciones: pendiente, aprobada o rechazada. Se guardan aparte y no cambian el estado histórico por sí solas.

Cálculo: tarde cuenta como presencia; justificado cuenta como inasistencia para umbrales, pero se informa por separado.

Edición: si ya existe la clase, se edita; no se crea otra. El índice único es la red de seguridad.

Reportes: inasistencias usa ?minimo= como filtro (default 10%), no como regla legal. No confundas ese parámetro con la normativa.

Calificaciones y planilla

Notas: escala 1-10. TEA/TEP/TED son valoraciones cualitativas y no entran al promedio. Un texto no es un cero.

Periodos: P1 y P2 regulares; además hay preliminares, finales e intensificación. Solo las regulares numéricas promedian.

Aprobación: P1 y P2 aprobados con 7 o más; si P2 no llega, corresponde intensificación. La nota final es el promedio de P1/P2 si ambos están aprobados.

Intensificación: se aprueba con 4 o más; si aprueba, la nota final es la mejor nota de intensificación.

Nota final: la ventana global habilita la carga. Si hay dos finales de cuatrimestre, el promedio automático tiene prioridad sobre la nota manual.

Persistencia: al guardar se recalculan promedios y estados en alumno_promedios y alumno_materias_estado. No los edites a mano.

La planilla oficial

PlanillaExcel reproduce el diseño de papel: 35 filas de alumnos, 24 columnas (A..X), encabezado en las filas 11-14 y primera fila de alumno en la 15. Las notas se mapean por columna; las columnas U a X corresponden a intensificación (diciembre, febrero-marzo, marzo-abril y final).

PDF: vista Blade con DomPDF. Se usa para imprimir y archivar.

Excel: PhpSpreadsheet con estilos, anchos, validaciones y fórmula de pie. Es el archivo que se entrega a secretaría.

Guardar: POST /calificaciones/planilla/guardar procesa la planilla completa; una sola entrada de historial si es lote.

Cuidado: si cambiás el orden de columnas, actualizá el mapa de PlanillaExcel, la vista Blade y las pruebas de lectura.

Cursos, materias y horarios

Cursos: se identifican por anio + division; el año tope sale de ParametroDivision activo, con fallback 7.

Divisiones: parametro_divisiones define qué combinaciones existen. Activá años/divisiones antes de promocionar.

Materias: catálogo por año; el set oficial 2026 tiene 155 materias. No dupliques nombres al importar.

Horario: días 1-5 (Lun-Vie), slots h1..h9, grupo A/B/null. UNIQUE(curso_id, dia, slot_key, grupo) evita solapamientos.

Slots: SlotKey normaliza h1..h9 y acepta variantes; la grilla del front y la vista imprimible comparten el orden.

Impresión: GET /horarios/planilla recibe anio, division, grupo y ciclo; exige ámbito.

Profesores y suplencias

Ficha: profesor, email, asignaciones por año/división/materia y bajas. El vínculo con users es por email; si no coincide, el profesor no ve su ámbito.

Suplencias: cadena titular → suplente por materia y curso. Máximo 4 eslabones, sin ciclos, sin auto-reemplazo y sin duplicados.

Ámbito: un profesor ve sus cursos por profesor_asignaciones y también los que cubre como suplente.

Baja: forceDelete es definitivo; no hay papelera. Confirmá y registrá el motivo.

Documentos

Subida: máximo 50 MB; mimes permitidos: pdf, doc(x), xls(x), ppt(x), txt, csv, imágenes, zip y rar.

Almacenamiento: storage/app/documentos, disco local. La tabla guarda ruta, mime, tamaño y nombre original.

Visibilidad: roles JSON; null significa todos (documentos legacy). El creador siempre puede editar el suyo.

Descarga: pasa por el controlador; no expongas storage directamente. El rol se valida en cada request.

Backup: el backup de base no incluye los archivos. Copiá también storage/app/documentos.

Historial, dashboard y backups

Historial: activity_logs con acción legible, action_type, usuario y diff. Filtros por categoría, tipo, búsqueda y fecha; cache 300 s sin filtros.

Sello: SelloCambios mantiene un id por grupo en cache (clave sello:grupo:...) y lo actualiza ActivityLogger en cada escritura. Leer /datos/sello no consulta la base.

Dashboard: top performers y cantidad de preceptores; no reemplaza los listados oficiales.

Backup: mysqldump primero; si falla, JSON comprimido en ZIP. Timeout 300 s, throttle 20/60 min y confirmación de contraseña.

Archivos: los backups quedan en storage/app/backups y se envían por correo. Verificá que el archivo exista y tenga tamaño antes de confiar.

El cambio más peligroso no siempre es el más grande

Un cambio de una línea en el cálculo de promedios, en el unique de asistencias o en el ámbito de un rol puede afectar a toda la escuela. Si tocás una regla de negocio, escribí primero el caso de prueba y hacelo revisar por alguien que conozca la normativa.

09 Reglas de negocio

Lo que no se cambia sin entender el impacto.

Estas reglas están distribuidas entre middleware, controladores y clases de soporte. No están todas en un servicio único. Antes de modificarlas, buscá si ya existe una constante o un helper: el proyecto intenta tener una sola fuente de verdad por regla.

Roles y niveles

Rol	Nivel	Alcance
admin	5	Control total; único que puede gestionar a directivo/admin.
directivo	4	Control total, salvo tocar admin/directivo.
jefe_preceptores	3	Casi todo; no toca admin/directivo. Configura ventana de nota final.
preceptor	2	Lectura global; escritura en su ámbito; edita profesores asignados.
prosecretaria	2	Equivalente a preceptor en ámbito y permisos.
profesor	1	Solo calificaciones y lectura de sus cursos asignados/cubiertos.

Ámbito

Global: admin, directivo y jefe_preceptores tienen userCanWriteAll y userIsAssignedTo siempre verdadero.

Usuario: preceptor y prosecretaria se limitan a user_anio_division. División null significa todo el año.

Profesor: se vincula por email con profesores; suma profesor_asignaciones y las coberturas de suplencias.

Sin rol: nivel 0. No pasa los controles de lectura docente; recibe 403.

Ficha completa: solo admin/directivo/jefe ven DNI, estado y observaciones. Los demás reciben contacto y cursos.

Asistencia

Los estados y las fórmulas viven en AsistenciaReglas. La regla se aplica igual en historial, inasistencias y estadísticas: tarde cuenta como presencia; justificado cuenta como inasistencia para el umbral, pero se muestra por separado.

```
// Presencia: (presentes + tardanzas) / total
// Inasistencia: (ausentes + justificados) / total
$porcentaje = round((($presentes + $tardanzas) / $total) * 100, 2);
$inasistencia = round((($ausentes + $justificados) / $total) * 100, 2);
```

Una clase por día: UNIQUE(curso_id, fecha). Si la persona intenta tomar lista dos veces, el backend debe devolver un error claro, no un 500.

Estados de clase: dictada o suspendida. Una clase suspendida conserva constancia y comentario, pero no alumnos presentes.

Justificaciones: pendiente/aprobada/rechazada. No borres una justificación para "corregir": cambiá su estado y dejá historial.

Reporte: ?minimo= es un filtro de visualización (default 10), no una condición de promoción.

Calificaciones

Escala: 1 a 10. Solo valores numéricos positivos promedian; TEA/TEP/TED y textos se conservan para mostrar y no cuentan.

Periodos: P1 y P2 regulares. Preliminares, finales e intensificación se muestran, pero no entran al promedio regular.

Aprobación de periodo: 7 o más. Si P1 y P2 están aprobados, la nota final es el promedio de ambos.

Intensificación: se aprueba con 4 o más. Si aprueba, la nota final es la mejor nota de intensificación.

Final manual: si hay dos finales de cuatrimestre, el promedio automático tiene prioridad. No mezcles ambas fuentes sin revisar recalcularEstado.

Persistencia: los promedios y estados se recalculan después de cada cambio. Una edición directa en la tabla queda fuera de la historia y puede romper la vista.

Ventana de nota final

Configuración: fila única lógica; se toma la última. Tiene fecha_inicio, fecha_fin, ciclo_lectivo y alcance.

Alcance: p1, p2 o ambos. Sin valor se asume ambos por compatibilidad.

Quién: solo admin y jefe_preceptores (directivo pasa por jerarquía). Cerrar la ventana es inmediato.

Efecto: si la ventana está cerrada, el backend devuelve 403 al intentar cargar una nota final. El frontend deshabilita el campo, pero la validación real es del servidor.

Promoción y ciclo

Año tope: SchoolYear::maxYear toma el máximo activo de ParametroDivision y cae en 7 si no hay datos.

Preview: simula y devuelve promoción, egresos y faltantes. No modifica nada.

Aplicar: en transacción: primero egresa a los que están en el tope, después sube al resto. Así el anteúltimo año no egresa por error.

Reversión: sentido down baja un año y recupera egresados a su último curso. También exige confirmación.

Contraseña: las operaciones masivas y destructivas exigen confirmación reciente (5 minutos) vía /auth/confirm-password.

Habilitar años: crea ParametroDivision y, si activate_courses, los cursos faltantes. Es idempotente por updateOrCreate/firstOrCreate.

Suplencias y backups

Suplencias: máximo 4 eslabones; sin ciclos, auto-reemplazo ni duplicados. La cadena se arma sola y se valida antes de guardar.

Backup: solo admin/directivo/jefe; contraseña reciente; throttle 20/60; mysqldump con timeout 300 s.

Fallback: si mysqldump falla, se exportan todas las tablas a JSON y se comprimen en ZIP. El email se envía igual, pero revisá el modo y el error.

Importación: es destructiva. Tratála como una operación de producción: backup previo, ventana de mantenimiento y verificación posterior.

> **La normativa manda sobre el código**
> Las reglas de aprobación, intensificación y promoción responden a la normativa escolar. Si el código y la normativa no coinciden, no "arregles" el código para que pase una prueba: documentá el caso, consultá y recién después cambiá la regla con tests.

10 Pruebas y calidad

Qué se puede verificar antes de publicar.

El proyecto tiene pruebas unitarias y de feature, pero no cubre toda la aplicación. La estrategia realista es ejecutar lo existente, agregar una prueba por cada regla que toques y completar con una verificación manual de las pantallas afectadas.

Pruebas existentes

Archivo	Qué protege
Unit/RoleHierarchyTest	Niveles, isAtLeast y canManage.
Unit/SuplenciasTest	Ciclos, profundidad y validación de eslabones.
Unit/ActivityDiffTest	Diff de cambios y descarte de campos intactos.
Unit/PaginacionTest	per_page inválido, negativo y tope 200.
Unit/CalificacionesLecturaTest	Lectura de TEA/TEP/TED sin convertirlos en 0.
Unit/EmailInstitucionalTest	Validación del dominio institucional.
Unit/RouteConfigTest	Throttle de registro y mensajes en español.
Feature/EnsureRoleTest	Jerarquía del middleware role.
Unit/ExampleTest	Prueba base del framework.

Comandos

```
# Backend (Docker)
docker compose exec backend php artisan test
docker compose exec backend ./vendor/bin/pint --test

# Frontend (Docker)
docker compose exec frontend npm run lint
docker compose exec frontend npm run build
```

En la máquina de desarrollo, sin Docker y con los binarios fuera del PATH, la verificación de un lote es siempre la misma secuencia. Ningún cambio se da por terminado con un solo paso.

```
# 1) Sintaxis de cada archivo PHP tocado
php.exe -l app/Dominios/Alumnos/AlumnoController.php

# 2) Lint del frontend, sin errores nuevos respecto de la linea base
node.exe node_modules/eslint/bin/eslint.js src --quiet

# 3) Build real de produccion
node.exe node_modules/vite/bin/vite.js build

# 4) Prueba en el navegador, con datos y rol reales
```

php -l: no reemplaza un test, pero corta el error de sintaxis antes de que llegue a la pantalla de alguien.

eslint --quiet: solo muestra errores; el proyecto tiene avisos históricos, así que lo que importa es que no aparezcan errores nuevos.

vite build: detecta imports roto, variables de entorno mal nombradas y errores de JSX que el dev server perdona.

Prueba real: abrí la pantalla, hacé la acción y mirá la consola. Un build verde no demuestra que la planilla se vea bien ni que el Excel baje con el nombre correcto.

La base de tests es SQLite en memoria

phpunit.xml usa sqlite :memory: y los tests actuales no consultan la base. Las migraciones tienen SQL específico de MySQL, así que no agregues RefreshDatabase sin portar el esquema. Para probar persistencia real, usá una base MySQL temporal y datos ficticios.

Qué probar si tocás...

Cambio	Verificación mínima
Permisos o ámbito	Login con cada rol; intentar leer y escribir fuera del curso; confirmar 403.
Asistencia	Tomar lista, editar la misma clase, justificar y consultar inasistencias.
Calificaciones	Cargar nota regular, TEA, final cerrada y verificar promedio/estado.
Planilla	Abrir PDF y Excel; comparar filas, columnas y totales con la pantalla.
Padrón en Excel	Bajar la lista con filtros puestos; abrir las dos hojas; revisar el nombre lista_alumnos-DD-MM-AAAA.xlsx y que las fechas no salgan como números.
Refresco automático	Editar en una segunda sesión y confirmar que la otra pantalla se actualice sola sin pisar un campo con cambios sin guardar.
Promoción	Ejecutar preview, revisar totales y aplicar en una base temporal.

Antes de decir "listo"

1. La app levanta y /api/ping responde.
2. Login, logout y restauración de sesión funcionan.
3. La pantalla afectada carga sin errores de consola ni 500.
4. El caso feliz y al menos un caso de error muestran mensajes claros.
5. Los permisos se probaron con un rol inferior.
6. Si hubo escritura, el historial registró la acción y el sello cambió.
7. Si hubo migración, se probó en una base limpia y en una con datos.
8. El build de producción termina sin errores.

Regresión mínima por módulo

Módulo	Caso que no puede fallar
Sesión	Login, restauración con /me, logout y cierre en otro equipo.
Alumnos	Buscar, abrir ficha, editar dentro del ámbito y egresar.
Padrón	Descargar la lista con 1200 alumnos: las dos hojas, el autofiltro y el nombre con la fecha.
Asistencia	Tomar lista, editar la clase y justificar una falta.
Calificaciones	Cargar regular, TEA y final con ventana abierta/cerrada.
Planilla	PDF y Excel con 35 filas, columnas alineadas y totales.
Cursos y horarios	Crear curso, mover una celda y abrir la planilla.
Profesores	Asignar materia y verificar que el docente vea su curso.
Documentos	Subir, descargar y comprobar visibilidad con otro rol.
Promoción	Preview y aplicación en base temporal; verificar egresos.

Qué no cubre la suite

No hay pruebas de integración contra MySQL: los tests no ejecutan migraciones ni tocan tablas reales.

No hay E2E de navegador: los guards, la caché y la impresión se validan a mano.

No hay pruebas de carga ni de concurrencia: dos usuarios editando la misma clase o planilla no están cubiertos.

No hay regresión visual: un cambio de CSS puede romper una tabla sin que falle un test.

La suite no reemplaza la revisión de normativa ni la prueba con un rol inferior.

Por eso el checklist de regresión por módulo es parte del trabajo, no un extra. Si automatizás un caso nuevo, preferí una prueba de regla pura antes que una prueba frágil de interfaz.

11 Despliegue y mantenimiento

De desarrollo a producción sin sorpresas.

El camino real de producción es un servidor Linux con Docker Compose; está paso a paso en docs/DESPLEGUE-LINUX.md. El compose levanta cuatro servicios: mysql 8 (solo loopback :3307), backend con PHP-FPM, frontend que compila Vite dentro de la imagen y sirve un nginx estático, y un nginx :8000 que es el único puerto que ve la escuela y hace de origen único (no hay CORS).

Qué ya viene resuelto en la imagen

OPcache: 256 MB con validate_timestamps=1 y revalidate_freq=2. Se deja la validación encendida porque el código va montado por volumen; con el código congelado en la imagen se puede pasar a 0, pero entonces cada despliegue tiene que reconstruir o reiniciar el contenedor.

PHP-FPM con pool propio: atiende varias peticiones en paralelo. php artisan serve atiende de a una y con varios preceptores a la vez forma cola; por eso el server va con Docker.

Nginx: gzip para JSON y JS/CSS, Cache-Control de un año con immutable en los assets hasheados y no-cache en index.html.

Subidas: upload_max_filesize y post_max_size en 64M (docker/backend/uploads.ini), coherentes con el límite de 50 MB de Documentos.

memory_limit: revisalo antes de exportar padrones grandes. Con 1.200 alumnos y el .xlsx del padrón, 128 M se queda corto: dejá 256 M o más.

Zona horaria: config/app.php usa America/Argentina/Buenos_Aires y el MySQL del compose también va en ese huso. Si el server está en UTC, la fecha del archivo y del padrón cambian de día a las 21:00 en vez de la medianoche.

> **Por qué NO se corre config:cache en este armado**
> El backend está montado por volumen contra el mismo repo que usa el desarrollo, así que bootstrap/cache/config.php congela el entorno del momento del cacheo (DB_HOST=mysql) y termina rompiendo un artisan serve local. Para activar config:cache y route:cache hay que separar los códigos: copiar backend/ dentro de la imagen en lugar de montarlo.

Checklist de producción

Área	Acción
Entorno	APP_ENV=production, APP_DEBUG=false, APP_KEY propia y HTTPS obligatorio al frente del nginx :8000.
Base de datos	Usuario con permisos mínimos, no root; MySQL sin puerto público; backup verificado antes de migrar.
Backend	composer install dentro del contenedor; migraciones con --force; memory_limit 256M.
Frontend	npm run build dentro de la imagen (docker/frontend/Dockerfile); se sirve dist/ con nginx, nunca el servidor de Vite.
Permisos	storage/ y bootstrap/cache/ escribibles por el usuario de PHP-FPM.
Archivos	php artisan storage:link y respaldo de storage/app/documentos.
Servicios	Quitar Mailpit y el 3307 del acceso público; SMTP real.
Seguridad	CORS solo con el dominio real; rate limits activos; logs fuera del webroot.

Secuencia de despliegue

```
# 0) Backup verificado ANTES de tocar nada (pantalla Inicio > Descargar backup
#    SQL, o mysqldump desde el contenedor). Sin backup no se migra.

# 1) Primera vez en el server
bash scripts/iniciar.sh
#  construir imagenes -> levantar -> esperar MySQL sano -> composer install
#  -> migrate --force -> storage:link -> verificar /api/ping
```

```
cp backend/.env.docker backend/.env
docker compose exec backend php artisan key:generate --force
docker compose exec backend php artisan migrate --seed --force
docker compose restart backend

# 2) Comprobar que esta SERVIDO, no solo levantado
docker compose ps
curl -s http://127.0.0.1:80/api/ping     # {"ok":true}

# 3) Actualizar el frontend (el bundle se quema en la imagen)
docker compose build frontend && docker compose up -d frontend

# 4) Actualizar el backend (codigo montado por volumen)
docker compose up -d
docker compose exec backend php artisan migrate --force

# 5) Apagar la bandeja de prueba cuando haya SMTP real
docker compose up -d --scale mailpit=0
```

El firewall del server abre solo el puerto público (80 o 8000 según el compose). MySQL, Mailpit y el resto quedan en loopback o en la red interna de Docker.

Si preferís no usar el script, los pasos son los mismos en orden: build, up, esperar a MySQL, composer install --optimize-autoloader, migrate --force, storage:link y ping. Lo que no se saltea es el backup y la prueba de login desde otro equipo de la red.

Actualización y rollback

1. Avisar la ventana y poner el sistema en mantenimiento si la migración lo requiere.
2. Descargar y verificar el backup de base y de documentos.
3. Desplegar código y dependencias; ejecutar migraciones.
4. Compilar el frontend y limpiar cachés.
5. Probar ping, login, una pantalla de lectura y una escritura controlada.
6. Si falla, volver a la versión anterior del código y, si la base cambió, restaurar el backup.

No todas las migraciones se pueden revertir con migrate:rollback. Si la migración borra o transforma datos, el rollback no recupera la información: la única vuelta atrás confiable es el backup.

Backups y archivos

Base: mysqldump (principal) o JSON+ZIP (contingencia). Se guardan en storage/app/backups y se envían por correo.

Documentos: storage/app/documentos no entra en el dump. Copiá la carpeta con rsync/robocopy y verificá tamaños.

Frecuencia: definí una política (diaria/semanal) y probá una restauración. Un backup que nunca se restauró no es un backup.

Retención: no borres el último backup bueno hasta que el nuevo esté verificado.

Secretos: el .env no se versiona; guardá una copia segura de APP_KEY y credenciales.

Logs y mantenimiento

Aplicación: storage/logs/laravel.log. Revisá errores 500 y lentitud; LogSlowRequests ayuda a detectar requests pesadas.

Servidor: logs de Nginx y PHP-FPM. Un 502 suele ser PHP-FPM caído, no Laravel.

Caché: si cambiás configuración y no ves efecto, ejecutá config:clear y volvé a cachear.

Sesiones: si un usuario no puede entrar, revisá status, api_sesiones y el límite de 10 equipos.

Correo: si no llega el recupero, revisá MAIL_* y la bandeja de Mailpit en desarrollo.

Dependencias: actualizá con prueba en una rama y build reproducible; no actualices Laravel y React a la vez sin necesidad.

Nunca copies el .env de desarrollo a producción

APP_DEBUG=true expone trazas y configuración; Mailpit no entrega correos reales; MySQL con root y puerto abierto es un riesgo. Producción necesita su propio .env, su propia base y HTTPS.

12 Depuración

Síntomas frecuentes y por dónde empezar.

La mayoría de los incidentes se resuelve mirando el status HTTP, el mensaje del backend y el estado de la sesión. Antes de reiniciar todo, identificá en qué capa se corta: navegador, red, Laravel o MySQL.

Tabla de diagnóstico

Síntoma	Causa probable	Qué revisar
401 en todo	Token ausente, borrado o usuario inactivo.	localStorage, /api/me, status y api_sesiones.
403 en una pantalla	Rol o ámbito insuficiente.	role de la ruta, user_anio_division, asignaciones y suplencias.
Lista vacía para profesor	Email de users distinto de profesores o sin asignaciones.	profesores.email, profesor_asignaciones, coberturas.
429 al entrar	Throttle de login (5/min por email+IP).	Esperar; no reintentar en loop; revisar logs.
No guarda notas	Ventana cerrada, nota final o ámbito.	Mensaje 403, ventana_nota_final, permitePeriodo.
No toma asistencia	Ya existe clase para curso y fecha.	UNIQUE(curso_id, fecha); editar la clase existente.
PDF/Excel falla	Vista, extensión PHP o memoria.	laravel.log, zip/gd, memory_limit, cantidad de filas.
El Excel baja vacío o con nombre raro	Filtros puestos, respuesta leída como texto o nombre armado en el backend.	apiBlob, descargarBlob, Content-Disposition dual y la consulta con filtros.
La pantalla no se renueva sola	Sello, sesión o pestaña inactiva.	GET /api/datos/sello, clave sello:grupo:<grupo> en cache, polling de la pestaña.
Backup falla	mysqldump ausente, permisos o timeout.	storage/app/backups, modo SQL/JSON, error del proceso.
Frontend viejo	Build no actualizado o caché de pantalla obsoleta.	Versión de esquema, index.html sin caché, npm run build.

Secuencia de diagnóstico

1. Reproducí el problema y anotá la pantalla, el rol y la hora.
2. Abrí DevTools → Network: mirá status, URL, payload y respuesta.
3. Si es 401/403, revisá sesión y permisos antes de tocar la lógica.
4. Si es 422, leé data.errors: suele ser validación, no un bug.
5. Si es 500, buscá la excepción en storage/logs/laravel.log.
6. Si la API responde bien pero la pantalla no, revisá la consola y el estado de React.
7. Si los datos no coinciden, compará la consulta SQL con la tabla y el historial.
8. Escribí la causa y la solución; si es recurrente, agregá una prueba.

Comandos útiles

```
docker compose logs -f backend nginx frontend
docker compose exec backend php artisan route:list
docker compose exec backend php artisan about
docker compose exec backend php artisan tinker
docker compose exec backend tail -n 100 storage/logs/laravel.log
```

No "arregles" con un reinicio lo que no entendés

Reiniciar puede borrar la evidencia. Si el problema es intermitente, primero capturá el request, el status y el log. Después reiniciá. Y nunca borres cachés de producción sin avisar: puedes provocar una caída momentánea.

13 Cómo contribuir

Convenciones y checklist de cambio.

La mejor contribución es la que se entiende sin preguntar. En este proyecto eso significa respetar el dominio, reutilizar el soporte existente y dejar rastro en el historial. No hace falta que el código sea clever; hace falta que sea predecible.

Antes de escribir código

Leé docs/MAPA_SISTEMA.md y docs/INSTALACION.md.

Buscá si el problema ya tiene un helper, una constante o un endpoint.

Identificá la tabla, el controlador y la pantalla que vas a tocar.

Verificá qué roles pueden ejecutar la acción y cuál es el ámbito.

Si la regla es normativa, confirmá el criterio con dirección antes de codificar.

Convenciones de cambio

Área	Criterio
Backend	Controlador en el dominio; validación explícita; AuthorizesScope; transacción si toca varias tablas.
Base de datos	Migración nueva; no editar la aplicada; índices para consultas nuevas; FK explícita.
Frontend	Módulo existente; apiGet/apiPost para datos y apiBlob + descargarBlob para archivos; keys de caché claras; useAvisoDeCambios si la pantalla se renueva sola; estados de carga/error/vacío.
Auditoría	ActivityLogger en escrituras relevantes; action_type estable y legible.
Pruebas	Una prueba por regla nueva o bug corregido; si no se puede, documentar la verificación manual.
Texto	Español claro para el usuario; mensajes de error sin códigos ni jerga.

Checklist de una contribución

1. El cambio resuelve un problema concreto y tiene un alcance acotado.
2. No rompe permisos ni el ámbito de un rol inferior.
3. La migración se probó en una base limpia y con datos.
4. El backend devuelve mensajes claros y registra historial.
5. El frontend invalida la caché correcta y no duplica la capa de red.
6. Las pruebas y el build pasan.
7. La verificación manual cubre el caso feliz y un error.
8. El cambio se documenta (qué, por qué y cómo revertir).

Lo que no hay que hacer

No tocar _backup_*/, base-de-datos/ ni backups/: son respaldos y no hay git que los reemplace.

No hardcodear roles, años máximos ni IDs de materias.

No confiar en el frontend para autorizar: el backend siempre valida.

No poner el token en una URL, un log o un mensaje de error.

No editar una migración ya aplicada en producción.

No ejecutar resets, importaciones o promociones en la base real sin backup y confirmación.

No subir .env, backups, documentos de alumnos ni datos personales al repositorio.

No volver a agregar un botón ni un cartel de "hay novedades": el refresco ya es automático y silencioso.

No escribir en la base de desarrollo fuera de una transacción con rollback en los harness de tinker.

No cambiar reglas de notas o asistencia sin revisar la normativa y los tests.

Definición de terminado

Un cambio está terminado cuando otra persona puede entenderlo, probarlo y revertirlo sin tu ayuda. Si necesitás explicar demasiado para que funcione, probablemente el diseño se pueda simplificar.

Glosario técnico

Términos que aparecen en el código, en la API o en la documentación. No es un glosario de Laravel ni de React: es el vocabulario propio del Sistema ENRO.

Término	Significado
Ámbito	Cursos o divisiones sobre los que un usuario puede operar.
ApiSesion	Registro de un equipo conectado; guarda token, IP y último uso.
apiBlob	Petición que devuelve el archivo como Blob con el token en la cabecera; se guarda con descargarBlob.
AuthToken	Middleware que resuelve el usuario desde Bearer o ticket de impresión.
Clase	Toma de asistencia de un curso en una fecha; puede ser dictada o suspendida.
Diff	Diferencia entre valores anteriores y nuevos que guarda el historial.
DomPDF	Librería que convierte vistas Blade en PDF.
Guard	Componente React que protege una ruta según sesión o rol.
Jefe de preceptores	Rol nivel 3; gestiona casi todo salvo admin/directivo.
Lote	Operación que afecta varios alumnos en una sola petición e historial.
Page cache	Copia en memoria/localStorage que evita pedir de nuevo al volver a una pantalla.
Padrón	Lista completa de alumnos en Excel: hoja PADRÓN (23 columnas) y hoja RESUMEN.
PhpSpreadsheet	Librería que genera los .xlsx reales: la planilla oficial y el padrón de alumnos.
Planilla oficial	Diseño de 35 filas por 24 columnas usado por secretaría.
Refresco automático	Sondeo del sello cada 12 s que renueva la pantalla sola, sin botón y sin pisar cambios sin guardar.
Scope	Restricción de una consulta al ámbito del usuario.
Sello	Número por grupo de datos; cambia con cada escritura y es lo que consulta el refresco automático.
Suplencia	Cadena titular → suplente por materia y curso; máximo 4 eslabones.
Ticket de impresión	Token temporal de 60 segundos para abrir una vista imprimible sin exponer el token real.

Anexos de referencia

Tres listas para tener a mano: comandos, archivos que conviene leer primero y una checklist de primer día. No reemplazan la lectura del código, pero ahorran tiempo.

A. Comandos

Comando	Uso
docker compose up -d	Levantar el entorno de desarrollo.
docker compose logs -f backend	Seguir errores de Laravel en vivo.
php -l archivo.php	Sintaxis antes de seguir editando (en Laragon, con la ruta completa del exe).
php artisan test	Ejecutar PHPUnit.
php artisan tinker ruta/archivo.php	Harness de verificación, siempre con transacción y rollback.
./vendor/bin/pint --test	Verificar estilo PHP sin modificar.
npm run lint	Ejecutar ESLint.
npm run build	Compilar el frontend para producción.
php artisan migrate:status	Ver migraciones aplicadas y pendientes.
php artisan route:list	Listar rutas, middleware y controladores.
php artisan config:clear	Limpiar caché de configuración.

B. Archivos para leer primero

Archivo	Por qué
docs/MAPA_SISTEMA.md	Pantalla → API → backend → tabla.
docs/INSTALACION.md	Puesta en marcha y problemas comunes.
docs/DESPLEGUE-LINUX.md	El despliegue real en el servidor, con Docker.
backend/routes/api.php	Contrato real de la API y grupos de rol.
backend/app/Support/AuthorizesScope.php	Cómo se decide el acceso fino.
backend/app/Support/RoleHierarchy.php	Niveles y gestión de usuarios.
backend/app/Support/PadronExcel.php	Las 23 columnas del padrón y la hoja RESUMEN.
backend/app/Support/PlanillaExcel.php	Diseño oficial y mapa de columnas.
frontend/src/compartido/lib/api.js	Red, errores, caché e invalidación.
frontend/src/compartido/hooks/useAvisoDeCambios.js	El contrato del refresco automático.
frontend/src/App.jsx	Rutas, guards y carga diferida.

C. Primer día

1. Levantá el entorno y verificá /api/ping, frontend y Mailpit.
2. Entrá con el admin inicial; cambiá la clave si se generó una temporal.
3. Recorré Dashboard, Alumnos y Asistencia con un rol de prueba.
4. Leé api.php, AuthorizesScope y api.js completos.
5. Elegí un bug chico y seguí el recorrido pantalla → ruta → controlador → tabla.
6. Agregá o ajustá una prueba; ejecutá test, lint y build.
7. Documentá lo que aprendiste en el issue o en la descripción del cambio.

Licencia

El código del Sistema ENRO y esta documentación se distribuyen bajo los términos de la Licencia Pública General de GNU (GNU GPL), versión 3.0 o, a tu elección, cualquier versión posterior, publicada por la Free Software Foundation. Es software libre: podés usarlo, estudiarlo, modificarlo y compartirlo.

Qué implica el copyleft para quien mantiene el sistema

Una versión modificada sigue siendo GPL. Si desplegás un derivado en otra escuela o entregás una variante, tenés que entregar el código fuente de esos cambios bajo la misma licencia.

Sin restricciones adicionales. No se puede sumar una condición que quite las libertades de la GPL, ni cerrar el código de un derivado.

Constancia de los cambios. Marcá qué modificaste y cuándo, además de conservar los avisos de autoría y de licencia.

Archivos nuevos. Cuando agregás un archivo, declará la licencia en el encabezado; la forma que leen las herramientas es una línea como SPDX-License-Identifier: GPL-3.0-or-later.

Dependencias

Las bibliotecas principales son de licencia permisiva (MIT en su mayoría: Laravel, React, Vite, PhpSpreadsheet y las utilidades de Node), compatible con GPL v3.0. MySQL se ejecuta como un programa aparte y no se enlaza con el código de la aplicación. Antes de sumar una dependencia nueva revisá su licencia: una AGPL o una propietaria cambia las condiciones con las que se distribuye el producto.

Sin garantía

El programa se distribuye con la esperanza de que sea útil, pero sin ninguna garantía; ni siquiera la garantía implícita de comercialización o de idoneidad para un propósito particular. Quien opera el sistema asume la responsabilidad sobre los respaldos, las migraciones y el uso que le da.

Los datos no son software

La GPL alcanza al código y a la documentación, no a los datos de la escuela. Los datos de alumnos y familias son de la institución y están sujetos a la normativa de protección de datos personales (en Argentina, la Ley 25.326): no se comparten con el código, no van en un repositorio público y no se publican junto con el sistema.

Texto completo de la licencia

La versión íntegra de la GNU GPL v3.0 está publicada por la Free Software Foundation en https://www.gnu.org/licenses/gpl-3.0.html. Deberías haber recibido una copia junto con el sistema; si no fue así, consultala ahí o pedila.

Copyright (C) 2026 E.E.S.T. N° 2 - ISFT N° 38 - Equipo de desarrollo del Sistema ENRO

Fin del manual

Sistema ENRO · Manual del programador · Edición noviembre 2026